VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"JnanaSangama", Belgaum -590014, Karnataka.

LAB REPORT

on

OPERATING SYSTEMS

Submitted by

RIDHI GANESH(1WA24CS229)

in partial fulfilment for the award of the degree of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING

B.M.S. COLLEGE OF ENGINEERING

(Autonomous Institution under VTU)

BENGALURU-560019

Feb-2026 to June-2026

B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering

CERTIFICATE

This is to certify that the Lab work entitled "OPERATING SYSTEMS – 23CS4PCOPS" carried out by RIDHI GANESH(1WA24CS229), who is Bonafide student of B. M. S. College of Engineering. It is in partial fulfilment for the award of Bachelor of Engineering in Computer Science and Engineering of the Visvesvaraya Technological University, Belgaum during the year Feb-2026 to June-2026
. The Lab report has been approved as it satisfies the academic requirements in respect of a OPERATING SYSTEMS - (23CS4PCOPS) work prescribed for the said degree.

Dr. Seema Patil	Dr. Kavitha Sooda
Assistant Professor	Professor and Head
Department of CSE	Department of CSE
BMSCE, Bengaluru	BMSCE, Bengaluru

Index Sheet

Sl. No.	Experiment Title	Page No.

1.	Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time. a) FCFS b) SJF (Pre-emptive and Non-Pre-emptive) c) Priority (Pre-emptive and Non-Pre-emptive) d) Round Robin (Experiment with different quantum sizes for RR algorithm)	1-18
2.	Write a C program to simulate multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories – system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue.	19-22
3.	Write a C program to simulate Real-Time CPU Scheduling algorithms: a) Rate- Monotonic b) Earliest-deadline First c) Proportional scheduling	23-29
4.	Write a C program to simulate: a) Producer-Consumer problem using semaphores. b) Dining-Philosopher's problem	30-34
5.	Write a C program to simulate: a) Bankers' algorithm for the purpose of deadlock avoidance. b) Deadlock Detection	35-39
6.	Write a C program to simulate the following contiguous memory allocation techniques. a) Worst fit b) Best-fit c) First-fit	40-42
7.	Write a C program to simulate page replacement algorithms. a) FIFO b) LRU c) Optimal	43-47

Course Outcomes:

C01	Apply the different concepts and functionalities of Operating System
C02	Analyse various Operating system strategies and techniques
C03	Demonstrate the different functionalities of Operating System.
C04	Conduct practical experiments to implement the functionalities of Operating system.

Program 1: Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time.

a).FCFS:

```
 #include <stdio.h>


int main() {

    int n, bt[10], wt[10], tat[10];

    float awt = 0, atat = 0;


    printf("Enter number of processes: ");

    scanf("%d", &n);


    for(int i=0;i<n;i++) {

        printf("BT of P%d: ", i+1);

        scanf("%d",&bt[i]);

    }


    wt[0] = 0;


    for(int i=1;i<n;i++)

        wt[i] = wt[i-1] + bt[i-1];


    for(int i=0;i<n;i++) {

        tat[i] = wt[i] + bt[i];

    }


    printf("\nP\tBT\tWT\tTAT\n");
```

```
    for(int i=0;i<n;i++)

        printf("P%d\t%d\t%d\t%d\n",i+1,bt[i],wt[i],tat[i]);


    printf("\nAverage WT = %.2f",awt/n);

    printf("\nAverage TAT = %.2f",atat/n);


    return 0;
}
```

Output:

```
"D:\RIDHI OS\FCFS(LAB1).e
Enter number of processes (max 20): 3

Process P1
Arrival Time: 2
Burst Time: 4

Process P2
Arrival Time: 3
Burst Time: 5

Process P3
Arrival Time: 6
Burst Time: 4

P       AT      BT      CT      TAT     WT      RT
P1      2       4       6       4       0       0
P2      3       5       11      8       3       3
P3      6       4       15      9       5       5

Average WT = 2.67
Average TAT = 7.00
Average RT = 2.67

Process returned 0 (0x0)   execution time : 22.443 s
Press any key to continue.
```

b) SJF

1. Preemptive:

```
#include <stdio.h>
```

```
int main(){
  int n, completed = 0, curr_time = 0, pid[10], at[10], bt[10],
remt[10],
         ct[10], tat[10], wt[10], st[10], rt[10], i, j;
  double tat_sum = 0, wt_sum = 0;
  printf("Enter no. of processes: ");
  scanf("%d", &n);
  for (i = 0; i < n; i++) {
    pid[i] = i + 1;
    printf("Enter arrival time for process[%d]: ", i + 1);
    scanf("%d", &at[i]);
    printf("Enter burst time for process[%d]: ", i + 1);
    scanf("%d", &bt[i]);
    remt[i] = bt[i];
  }
  while (completed != n) {
    int min = 99999999,
        s = -1;
    for (i = 0; i < n; i++) {
      if (at[i] <= curr_time && remt[i] > 0 && remt[i] < min) {
        min = remt[i];
        s = i;
      }
    }
    if (s == -1) {
      curr_time++;
      continue;
```

```
    }
    if (remt[s] == bt[s])
      st[s] = curr_time;
    remt[s]--;
    if (remt[s] == 0) {
      ct[s] = curr_time + 1;
      tat[s] = ct[s] - at[s];
      tat_sum += tat[s];
      wt[s] = tat[s] - bt[s];
      wt_sum += wt[s];
      rt[s] = st[s] - at[s];
      completed++;
    }
    curr_time++;
  }
  printf("\nP\tAT\tBT\tCT\tTAT\tWT\tRT\n\n");
  for (j = 1; j < n + 1; j++) {
    for (i = 0; i < n; i++) {
      if (pid[i] == j)
        printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n", pid[i], at[i], bt[i],
ct[i],
               tat[i], wt[i], rt[i]);
    }
  }
  printf("\nAverage TAT: %.2f\n", (tat_sum / n));
  printf("Average WT: %.2f\n", (wt_sum / n));
}
```

Output:

```
"D:\RIDHI OS\SJF PREMTIV
Enter number of processes: 3

Process P1
Arrival Time: 6
Burst Time: 7

Process P2
Arrival Time: 8
Burst Time: 65

Process P3
Arrival Time: 6
Burst Time: 54

P       AT      BT      CT      TAT     WT      RT
P1      6       7       13      7       0       0
P2      8       65      132     124     59      59
P3      6       54      67      61      7       7

Average WT = 22.00
Average TAT = 64.00
Average RT = 22.00

Process returned 0 (0x0)   execution time : 7.459 s
Press any key to continue.
```

2. Non-Preemptive:

```
#include<stdio.h>

#include<stdlib.h>

#define max(a,b) ((a>b)? (a):(b))

int main(){

    int a=4,b=6,temp,comp_count=0;

    int sjf[a][b];

    printf("Enter the Process, Arrival Time and Burst Time:\n");

    for(int i=0; i<a; i++){

        for(int j=0; j<3; j++){
```

```
            scanf("%d",&sjf[i][j]);
        }
    }
    for(int i=0; i<a-1; i++){
        for(int j=0; j<a-i-1; j++){
            if(sjf[j][1] > sjf[j+1][1]){
                for(int k=0; k<3; k++){
                    temp = sjf[j][k];
                    sjf[j][k] = sjf[j+1][k];
                    sjf[j+1][k] = temp;
                }
            }
        }
    }
    int curr_time =0, is_comp[4] = {0};
    while(comp_count < a){
        int best_index = -1, min_burst = 9999;

        for(int i=0; i<a; i++){
            if(sjf[i][1] <= curr_time && is_comp[i] == 0){
                if(sjf[i][2] < min_burst) {
            min_burst = sjf[i][2];
            best_index = i;
        }
    }
}
if(best_index != -1){
```

```
    curr_time += sjf[best_index][2];

            sjf[best_index][3] = curr_time;


            sjf[best_index][4] = sjf[best_index][3] -
sjf[best_index][1];

            sjf[best_index][5] = sjf[best_index][4] -
sjf[best_index][2];

        is_comp[best_index] = 1;

        comp_count++;

    }

    else{

        curr_time++;

    }

}

printf("Process\tAT\tBT\tCT\tTAT\tWT\n");

    for(int i=0; i<a; i++){

        for(int j=0; j<b; j++){

            printf("%d\t",sjf[i][j]);

        }

        printf("\n");

    }

    int sumTAT =0, sumWT=0;

    for(int i=0; i<a; i++){

        sumTAT += sjf[i][4];

        sumWT += sjf[i][5];

    }

    printf("Average TAT: %2f",(float)sumTAT/a);
```

```
    printf("Average WT: %2f",(float)sumWT/a);
}
```

Output:

```
Process P1
Arrival Time: 2
Burst Time: 5

Process P2
Arrival Time: 3
Burst Time: 6

Process P3
Arrival Time: 1
Burst Time: 7

Process P4
Arrival Time: 3
Burst Time: 8

P       AT      BT      CT      TAT     WT      RT
P1      2       5       13      11      6       6
P2      3       6       19      16      10      10
P3      1       7       8       7       0       0
P4      3       8       27      24      16      16

Average WT = 8.00
Average TAT = 14.50
Average RT = 8.00

Process returned 0 (0x0)   execution time : 15.571 s
Press any key to continue.
```

c). Priority (Preemptive and Non-Preemptive)

1. Preemptive:

```
#include <stdio.h>
int main() {
    int n, i, time = 0, completed = 0;
    int bt[20], at[20], pr[20], rt[20];
    int wt[20], tat[20], ct[20];
    int highest, min_pr;
    int total_wt = 0, total_tat = 0;
    printf("Enter number of processes: ");
```

```
scanf("%d", &n);
for(i = 0; i < n; i++) {
    printf("\nProcess %d\n", i + 1);
    printf("Enter Arrival Time: ");
    scanf("%d", &at[i]);
    printf("Enter Burst Time: ");
    scanf("%d", &bt[i]);
    printf("Enter Priority: ");
    scanf("%d", &pr[i]);
    rt[i] = bt[i];
}
while(completed < n) {
    highest = -1;
    min_pr = 9999;
    for(i = 0; i < n; i++) {
        if(at[i] <= time && rt[i] > 0 && pr[i] < min_pr) {
            min_pr = pr[i];
            highest = i;
        }
    }
    if(highest == -1) {
        time++;
        continue;
    }
    rt[highest]--;
    time++;
    if(rt[highest] == 0) {
        completed++;
        ct[highest] = time;
```

```c
            tat[highest] = ct[highest] - at[highest];
            wt[highest] = tat[highest] - bt[highest];
            total_wt += wt[highest];
            total_tat += tat[highest];
        }
    }
    printf("\nPID\tAT\tBT\tPR\tCT\tWT\tTAT\n");
    for(i = 0; i < n; i++) {
        printf("%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
        i + 1, at[i], bt[i], pr[i], ct[i], wt[i], tat[i]);
    }
    printf("\nAverage Waiting Time = %.2f", (float)total_wt/n);
    printf("\nAverage Turnaround Time = %.2f\n", (float)total_tat/n);
    return 0;
}
```

Output:

```
RIDHI GANESH
Enter number of processes: 5

Enter Process ID: 1
Enter Arrival Time: 0
Enter Burst Time: 3
Enter Priority: 5

Enter Process ID: 2
Enter Arrival Time: 2
Enter Burst Time: 2
Enter Priority: 3

Enter Process ID: 3
Enter Arrival Time: 3
Enter Burst Time: 5
Enter Priority: 2

Enter Process ID: 4
Enter Arrival Time: 4
Enter Burst Time: 4
Enter Priority: 4

Enter Process ID: 5
Enter Arrival Time: 6
Enter Burst Time: 1
Enter Priority: 1

PID     AT      BT      PR      CT      TAT     WT      RT
1       0       3       5       3       3       0       0
2       2       2       3       11      9       7       7
3       3       5       2       8       5       0       0
4       4       4       4       15      11      7       7
5       6       1       1       9       3       2       2
```

2. Non- Preemptive:

```c
#include <stdio.h>

struct Process {

    int pid;

    int burst;
```

```
    int priority;
    int waiting;
    int turnaround;
    int completion;
};
int main() {
    int n, i, j;
    struct Process p[20], temp;

    printf("Enter number of processes: ");
    scanf("%d", &n);
    for(i = 0; i < n; i++) {
        printf("\nProcess %d\n", i + 1);
        p[i].pid = i + 1;
        printf("Enter Burst Time: ");
        scanf("%d", &p[i].burst);
        printf("Enter Priority: ");
        scanf("%d", &p[i].priority);
    }
    for(i = 0; i < n - 1; i++) {
        for(j = i + 1; j < n; j++) {
            if(p[i].priority > p[j].priority) {
                temp = p[i];
                p[i] = p[j];
                p[j] = temp;
            }
        }
    }
    p[0].waiting = 0;
```

```
    for(i = 1; i < n; i++) {
        p[i].waiting = 0;
        for(j = 0; j < i; j++) {
            p[i].waiting += p[j].burst;
        }
    }
    for(i = 0; i < n; i++) {
        p[i].turnaround = p[i].burst + p[i].waiting;
        p[i].completion = p[i].turnaround;
    }
    float total_wt = 0, total_tat = 0;
    printf("\nPID\tBT\tPR\tWT\tTAT\tCT\n");
    for(i = 0; i < n; i++) {
        printf("%d\t%d\t%d\t%d\t%d\t%d\n",
               p[i].pid, p[i].burst, p[i].priority,p[i].completion,
               p[i].waiting, p[i].turnaround);
        total_wt += p[i].waiting;
        total_tat += p[i].turnaround;
    }
    printf("\nAverage Waiting Time = %.2f", total_wt / n);
    printf("\nAverage Turnaround Time = %.2f\n", total_tat / n);
    return 0;
}
```

Output:

```
RIDHI GANESH(PRIORITY PRE
3
4
5
6
10
Enter burst times:
8
2
4
1
6
5
1

Priority scheduling (Pre-Emptive):
PID     Prior   AT      BT      CT      TAT     WT      RT
P1      3       0       8       15      15      7       0
P2      4       1       2       17      16      14      0
P3      4       3       4       21      18      14      0
P4      5       4       1       22      18      17      0
P5      2       5       6       12      7       1       0
P6      6       6       5       27      21      16      0
P7      1       10      1       11      1       0       0

Average turnaround time: 13.71
Average waiting time: 9.86

Process returned 0 (0x0)   execution time : 37.870 s
Press any key to continue.
```

d). Round Robin (Experiment with different quantum sizes for RR algorithm)

```c
#include <stdio.h>

int main() {

    int n, tq;

    int pid[20], at[20], bt[20], rt[20];

    int wt[20] = {0}, tat[20] = {0}, ct[20];

    int queue[100];

    int front = 0, rear = 0;

    int visited[20] = {0};
```

```c
    int current_time = 0, completed = 0;
    float total_wt = 0, total_tat = 0;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    printf("Enter time quantum: ");
    scanf("%d", &tq);
    for(int i = 0; i < n; i++) {
        printf("\nProcess %d\n", i + 1);
        printf("Enter Process ID: ");
        scanf("%d", &pid[i]);
        printf("Enter Arrival Time: ");
        scanf("%d", &at[i]);
        printf("Enter Burst Time: ");
        scanf("%d", &bt[i]);
        rt[i] = bt[i];
    }
    while(completed < n) {
        for(int i = 0; i < n; i++) {
            if(at[i] <= current_time && visited[i] == 0) {
                queue[rear++] = i;
                visited[i] = 1;
            }
        }
        if(front == rear) {
            current_time++;
            continue;
        }
        int p = queue[front++];
        if(rt[p] > tq) {
```

```
            current_time += tq;
            rt[p] -= tq;
            for(int i = 0; i < n; i++) {
                if(at[i] <= current_time && visited[i] == 0) {
                    queue[rear++] = i;
                    visited[i] = 1;
                }
            }
            queue[rear++] = p;
        }
        else {
            current_time += rt[p];
            ct[p] = current_time;
            tat[p] = ct[p] - at[p];
            wt[p] = tat[p] - bt[p];
            rt[p] = 0;
            completed++;
        }
    }
    printf("\nPID\tAT\tBT\tCT\tWT\tTAT\n");
    for(int i = 0; i < n; i++) {
        printf("%d\t%d\t%d\t%d\t%d\t%d\n", pid[i], at[i], bt[i], ct[i],
wt[i], tat[i]);
        total_wt += wt[i];
        total_tat += tat[i];
    }
    printf("\nAverage Waiting Time = %.2f", total_wt / n);
    printf("\nAverage Turnaround Time = %.2f\n", total_tat / n);
    return 0;
```

```
}
```

Output:

```
RIDHI GANESH(ROUND ROBII
Enter Burst Time: 5

Enter Process ID: 2
Enter Arrival Time: 1
Enter Burst Time: 3

Enter Process ID: 3
Enter Arrival Time: 2
Enter Burst Time: 1

Enter Process ID: 4
Enter Arrival Time: 3
Enter Burst Time: 2

Enter Process ID: 5
Enter Arrival Time: 4
Enter Burst Time: 3

PID     AT      BT      CT      TAT     WT      RT
1       0       5       14      14      9       0
2       1       3       12      11      8       1
3       2       1       5       3       2       2
4       3       2       7       4       2       2
5       4       3       13      9       6       3

Average Waiting Time = 5.40
Average Turnaround Time = 8.20
Process returned 0 (0x0)   execution time : 52.971 s
Press any key to continue.
```

2. Write a C program to simulate multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories – system processes and user processes. System processes are to be

given higher priority than user processes. Use FCFS scheduling for the processes in each queue.

```
#include <stdio.h>
#include <string.h>
#define MAX 100
#define TQ 2
struct Process {
    char pid[10];
    int at, bt, rt, type;
    int ct, tat, wt;
    int done;
};
int main() {
    struct Process p[MAX];
    int n;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    for (int i = 0; i < n; i++) {
        printf("\nProcess %d\n", i + 1);
        printf("Enter PID: ");
        scanf("%s", p[i].pid);
        printf("Enter Arrival Time: ");
        scanf("%d", &p[i].at);
        printf("Enter Burst Time: ");
        scanf("%d", &p[i].bt);
        printf("Enter Type (1=System, 2=User): ");
        scanf("%d", &p[i].type);
        p[i].rt = p[i].bt;
```

```
        p[i].done = 0;
    }
    int time = 0, completed = 0;
    char ganttPID[MAX][10];
    int ganttTime[MAX];
    int gIndex = 0;
    int lastSysIndex = 0;
    while (completed < n) {
        int executed = 0;
        for (int i = 0; i < n; i++) {
            int idx = (lastSysIndex + i) % n;
            if (p[idx].type == 1 && p[idx].rt > 0 && p[idx].at <= time)
{
                ganttTime[gIndex] = time;
                strcpy(ganttPID[gIndex], p[idx].pid);
                gIndex++;
                if (p[idx].rt > TQ) {
                    time += TQ;
                    p[idx].rt -= TQ;
                } else {
                    time += p[idx].rt;
                    p[idx].rt = 0;
                    p[idx].ct = time;
                    p[idx].tat = p[idx].ct - p[idx].at;
                    p[idx].wt = p[idx].tat - p[idx].bt;
                    p[idx].done = 1;
                    completed++;
                }
                lastSysIndex = (idx + 1) % n;
```

```
                executed = 1;
                break;
            }
        }
        if (executed) continue;
        int earliest = -1;
        for (int i = 0; i < n; i++) {
            if (p[i].type == 2 && p[i].rt > 0 && p[i].at <= time) {
                if (earliest == -1 || p[i].at < p[earliest].at) {
                    earliest = i;
                }
            }
        }
        if (earliest != -1) {
            ganttTime[gIndex] = time;
            strcpy(ganttPID[gIndex], p[earliest].pid);
            gIndex++;
            int nextSysArrival = 1e9;
            for (int i = 0; i < n; i++) {
                if (p[i].type == 1 && p[i].rt > 0 && p[i].at > time) {
                    if (p[i].at < nextSysArrival)
                        nextSysArrival = p[i].at;
                }
            }
            if (time + p[earliest].rt <= nextSysArrival) {
                time += p[earliest].rt;
                p[earliest].rt = 0;

                p[earliest].ct = time;
```

```
                p[earliest].tat = p[earliest].ct - p[earliest].at;
                p[earliest].wt = p[earliest].tat - p[earliest].bt;
                p[earliest].done = 1;
                completed++;
            } else {
                int execTime = nextSysArrival - time;
                p[earliest].rt -= execTime;
                time = nextSysArrival;
            }
            continue;
        }
        time++;
    }
    ganttTime[gIndex] = time;
    printf("\n=========== MULTI-LEVEL QUEUE SCHEDULING ===========\n");
    printf("+------+------+----+----+----+-----+----+\n");
    printf("| PID  |TYPE | AT | BT | CT | TAT | WT |\n");
    printf("+------+------+----+----+----+-----+----+\n");
    float total_tat = 0, total_wt = 0;
    for (int i = 0; i < n; i++) {
        printf("| %-4s | %-4s | %-2d | %-2d | %-2d | %-3d | %-2d |\n",
               p[i].pid,
               p[i].type == 1 ? "SYS" : "USR",
               p[i].at,
               p[i].bt,
               p[i].ct,
               p[i].tat,
               p[i].wt);
        total_tat += p[i].tat;
```

```
        total_wt += p[i].wt;
    }
    printf("+------+------+----+----+----+-----+-----+\n");
    printf("\nAverage Turnaround Time = %.2f\n", total_tat / n);
    printf("Average Waiting Time    = %.2f\n", total_wt / n);
    printf("\n============ GANTT CHART ============\n\n");
    for (int i = 0; i < gIndex; i++) {
        printf("| %s ", ganttPID[i]);
    }
    printf("|\n");
    for (int i = 0; i < gIndex; i++) {
        printf("+----");
    }
    printf("+\n");
    for (int i = 0; i <= gIndex; i++) {
        printf("%-4d", ganttTime[i]);
    }
    printf("\n");
    return 0;
}
```

Output:

```
Enter number of System Processes: 3
System Process 1 (AT BT): 0 5
System Process 2 (AT BT): 8
6
System Process 3 (AT BT): 4 6

Enter number of User Processes: 2
User Process 1 (AT BT): 1 5
User Process 2 (AT BT): 7 6

--- Multi-Level Queue Scheduling ---

System (High Priority) Queue:
PID     AT      BT      CT      WT      TAT
1       0       5       5       0       5
2       8       6       14      0       6
3       4       6       20      10      16
Average WT = 3.33
Average TAT = 9.00

User (Low Priority) Queue:
PID     AT      BT      CT      WT      TAT
1       1       5       25      19      24
2       7       6       31      18      24
Average WT = 18.50
Average TAT = 24.00


...Program finished with exit code 0
Press ENTER to exit console.
```

3.Write a C program to simulate Real-Time CPU Scheduling algorithms:

a) Rate- Monotonic
b) Earliest-deadline First
c) Proportional scheduling

a). RMS:

```
#include <stdio.h>
#include <math.h>
int gcd(int a, int b) {
    if (b == 0)
        return a;
    return gcd(b, a % b);
}
int lcm(int a, int b) {
    return (a * b) / gcd(a, b);
}
int main() {
```

```
    int n;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    int Ci[n], Ti[n], remaining[n], original_id[n];
    for (int i = 0; i < n; i++) {
        printf("Enter execution time (C%d): ", i + 1);
        scanf("%d", &Ci[i]);
        printf("Enter period (T%d): ", i + 1);
        scanf("%d", &Ti[i]);
        remaining[i] = 0;
        original_id[i] = i + 1;
    }
    double U = 0;
    for (int i = 0; i < n; i++) {
        U += (double)Ci[i] / Ti[i];
    }
    double U_bound = n * (pow(2, (double)1/n) - 1);
    printf("\nCPU Utilization (U) = %.4f\n", U);
    printf("Utilization Bound (U_bound) = %.4f\n", U_bound);
    if (U > U_bound) {
        printf("Not Schedulable under RMS\n");
        return 0;
    }
    printf("Schedulable under RMS\n");
    int hyperperiod = Ti[0];
    for (int i = 1; i < n; i++) {
        hyperperiod = lcm(hyperperiod, Ti[i]);
    }
    printf("\nLCM (Hyperperiod) = %d\n", hyperperiod);
    for (int i = 0; i < n - 1; i++) {
        for (int j = i + 1; j < n; j++) {
            if (Ti[i] > Ti[j]) {
                int temp;
                temp = Ti[i]; Ti[i] = Ti[j]; Ti[j] = temp;
                temp = Ci[i]; Ci[i] = Ci[j]; Ci[j] = temp;
                temp = original_id[i]; original_id[i] = original_id[j];
original_id[j] = temp;
            }
        }
```

```
    }
    printf("\nRMS Table:\n");
    printf("Process\tExecution Time\tPeriod\n");
    for (int i = 0; i < n; i++) {
        printf("P%d\t%d\t\t%d\n", original_id[i], Ci[i], Ti[i]);
    }
    int schedule[hyperperiod];
    for (int t = 0; t < hyperperiod; t++) {

        for (int i = 0; i < n; i++) {
            if (t % Ti[i] == 0) {
                remaining[i] = Ci[i];
            }
        }
        int selected = -1;
        for (int i = 0; i < n; i++) {
            if (remaining[i] > 0) {
                selected = i;
                break;
            }
        }
        if (selected != -1) {
            schedule[t] = original_id[selected];
            remaining[selected]--;
        } else {
            schedule[t] = 0;
        }
    }
    printf("\nTimeline (Intervals):\n");
    int start = 0;
    for (int t = 1; t <= hyperperiod; t++) {
        if (t == hyperperiod || schedule[t] != schedule[start]) {
            if (schedule[start] == 0)
                printf("Time %d to %d: Idle\n", start, t);
            else
                printf("Time %d to %d: P%d\n", start, t,
schedule[start]);
            start = t;
        }
```

```c
    }
    return 0;
}
```

Output:

```
D:\WA24CS229\RMS.exe

Enter the number of processes: 2
Enter CPU burst times:
20 35
Enter the time periods:
50
100

LCM=100

Rate Monotone Scheduling:
PID     Burst   Period
1       20      50
2       35      100

0.750000 <= 0.828427 => true
Scheduling occurs for 100 ms

0ms onwards: Process 1 running
20ms onwards: Process 2 running
50ms onwards: Process 1 running
70ms onwards: Process 2 running
75ms onwards: CPU is idle

Process returned 0 (0x0)   execution time : 13.309 s
Press any key to continue.
```

b). EDF:

```c
#include <stdio.h>
#define MAX 10
int main() {
    int n;
    int burst[MAX], deadline[MAX], period[MAX];
    int remaining[MAX];
    int time = 0;
    printf("Enter number of tasks: ");
```

```
    scanf("%d", &n);
    for (int i = 0; i < n; i++) {
        printf("Task %d (burst deadline period): ", i + 1);
        scanf("%d %d %d", &burst[i], &deadline[i], &period[i]);
        remaining[i] = 0;  // initially no job released
    }
    printf("\nPID\tBurst\tDeadline\tPeriod\n");
    for (int i = 0; i < n; i++) {
        printf("%d\t%d\t%d\t\t%d\n", i + 1, burst[i], deadline[i],
period[i]);
    }
    int limit = 20;
    printf("\nScheduling occurs for %d ms\n\n", limit);

    while (time < limit) {
        for (int i = 0; i < n; i++) {
            if (time % period[i] == 0) {
                remaining[i] = burst[i];
            }
        }
        int min_deadline = 9999;
        int selected = -1;
        for (int i = 0; i < n; i++) {
            if (remaining[i] > 0) {
                if (deadline[i] < min_deadline) {
                    min_deadline = deadline[i];
                    selected = i;
                }
            }
        }
        if (selected == -1) {
            printf("%dms : CPU is idle.\n", time);
        } else {
            printf("%dms : Task %d is running.\n", time, selected + 1);
            remaining[selected]--;
        }
        time++;
    }
    return 0;
```

```
}
```

Output:

```
Enter number of processes: 3
Process 1 (arrival burst deadline period): 0 2 4 5
Process 2 (arrival burst deadline period): 0 3 7 20
Process 3 (arrival burst deadline period): 0 2 8 10

PID     Burst   Deadline        Period
1       2       4               5
2       3       7               20
3       2       8               10

Scheduling occurs for 20 ms

0ms : Task 1 is running.
1ms : Task 1 is running.
2ms : Task 2 is running.
3ms : Task 2 is running.
4ms : Task 2 is running.
5ms : Task 3 is running.
6ms : Task 3 is running.
7ms : CPU is idle.
8ms : CPU is idle.
9ms : CPU is idle.
10ms : CPU is idle.
11ms : CPU is idle.
12ms : CPU is idle.
13ms : CPU is idle.
14ms : CPU is idle.
15ms : CPU is idle.
16ms : CPU is idle.
17ms : CPU is idle.
```

c). Proportional Share:

```
#include<stdio.h>
#include <stdlib.h>
#include <time.h>
typedef struct
{
    char name[5]; int tickets;
}
Process;
int main() {
    int n, total_tickets = 0; float total_T =0.0;
    printf("Enter the number of Processes: ");
    scanf("%d", &n);
    Process p[n];
    srand(time(NULL));
    for (int i = 0; i < n; i++)
    {
        printf("\nProcess %d:\n", i + 1);
        sprintf(p[i].name, "P%d", i + 1);
        printf("Tickets: ");
        scanf("%d", &p[i].tickets);
```

```
        total_tickets += p[i].tickets;
        total_T +=p[i].tickets;
    }
    printf("\n--- Proportional Share Scheduling ---\n");
    printf("Enter the Time Period for scheduling: ");
    int m;

    scanf("%d",&m);
    for (int i = 0; i < m; i++)
    {
        int winning_ticket = rand() % total_tickets + 1;
        int accumulated_tickets = 0;
        int winner_index;
        for (int j = 0; j < n; j++)
        {
            accumulated_tickets += p[j].tickets;
            if (winning_ticket <= accumulated_tickets)
            {
                winner_index = j; break;
            }
        }
        printf("Tickets picked: %d, Winner: %s\n", winning_ticket,
        p[winner_index].name);
    }
    for (int i = 0; i < n; i++)
    {
        printf("\nThe Process: %s gets %0.2f%% of Processor Time.\n",
p[i].name,
        ((p[i].tickets / total_T) * 100));

    }
    return 0;
}
```

Output:

```
"C:\Users\hp\OneDrive\Deskt

RIDHI GANESH,1wa24cs229
Enter number of tasks: 3
Enter CPU share for each task:
50
30
20

CPU Allocation:
Task1 = 50%
Task2 = 30%
Task3 = 20%

Process returned 0 (0x0)   execution time : 17.487 s
Press any key to continue.
```

4. Write a C program to simulate:
a) Producer-Consumer problem using semaphores.
b) Dining-Philosopher's problem

a). Producer-Consumer problem using semaphores.

```c
#include <stdio.h>

#include <pthread.h>

#include <unistd.h>

#include <semaphore.h>

#define BUFFER_SIZE 5

int buffer[BUFFER_SIZE];

int in = 0, out = 0;

sem_t empty;

sem_t full;

pthread_mutex_t mutex;

void* producer(void* arg) {

    int item, i = 0;

    while (1) {
```

```
        item = i++;
        sem_wait(&empty);
        pthread_mutex_lock(&mutex);
        buffer[in] = item;
        printf("Produced: %d at buffer[%d]\n", item, in);
        in = (in + 1) % BUFFER_SIZE;
        pthread_mutex_unlock(&mutex);
        sem_post(&full);
        sleep(1);
    }
}
void* consumer(void* arg) {
    int item;
    while (1) {
        sem_wait(&full);
        pthread_mutex_lock(&mutex);
        item = buffer[out];
        printf("Consumed: %d from buffer[%d]\n", item, out);
        out = (out + 1) % BUFFER_SIZE;
        pthread_mutex_unlock(&mutex);
        sem_post(&empty);
        sleep(2);
    }
}
int main() {
    pthread_t prod, cons;
    sem_init(&empty, 0, BUFFER_SIZE);
    sem_init(&full, 0, 0);
    pthread_mutex_init(&mutex, NULL);
```

```
    pthread_create(&prod, NULL, producer, NULL);
    pthread_create(&cons, NULL, consumer, NULL);
    pthread_join(prod, NULL);
    pthread_join(cons, NULL);
    sem_destroy(&empty);
    sem_destroy(&full);
    pthread_mutex_destroy(&mutex);
    return 0;
}
```

Output:

```
"C:\Users\hp\OneDrive\Deskt

1. Produce
2. Consume
3. Exit
1
Produced Item 1

1. Produce
2. Consume
3. Exit
1
Produced Item 2

1. Produce
2. Consume
3. Exit
2
Consumed Item 2

1. Produce
2. Consume
3. Exit
3

Process returned 0 (0x0)   execution time : 19.179 s
Press any key to continue.
```

b) Dining-Philosopher's problem

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>
#define N 5
pthread_mutex_t forks[N];
```

```
pthread_t philosophers[N];
void* philosopher(void* num) {
    int id = *(int*)num;
    int left = id;
    int right = (id + 1) % N;
    while (1) {
        printf("Philosopher %d is thinking.\n", id);
        if (id % 2 == 0) {
            pthread_mutex_lock(&forks[left]);
            printf("Philosopher %d picked up left fork %d.\n", id,
left);
            pthread_mutex_lock(&forks[right]);
            printf("Philosopher %d picked up right fork %d.\n", id,
right);
        } else {
            pthread_mutex_lock(&forks[right]);
            printf("Philosopher %d picked up right fork %d.\n", id,
right);
            pthread_mutex_lock(&forks[left]);
            printf("Philosopher %d picked up left fork %d.\n", id,
left);
        }
        printf("Philosopher %d is eating.\n", id);
        sleep(2);
        pthread_mutex_unlock(&forks[left]);
        pthread_mutex_unlock(&forks[right]);
        printf("Philosopher %d put down forks %d and %d.\n", id, left,
right);
    }
}
int main() {
    int i;
    int ids[N];
    for (i = 0; i < N; i++) {
        pthread_mutex_init(&forks[i], NULL);
        ids[i] = i;
    }
    for (i = 0; i < N; i++) {
        pthread_create(&philosophers[i], NULL, philosopher, &ids[i]);
```

```
    }
    for (i = 0; i < N; i++) {
        pthread_join(philosophers[i], NULL);
    }
    for (i = 0; i < N; i++) {
        pthread_mutex_destroy(&forks[i]);
    }
    return 0;
}
```

Output:

```
"C:\Users\hp\OneDrive\Deskt
Philosopher 0 is Thinking
Philosopher 0 is Eating
Philosopher 1 is Thinking
Philosopher 1 is Eating
Philosopher 2 is Thinking
Philosopher 2 is Eating
Philosopher 3 is Thinking
Philosopher 3 is Eating
Philosopher 4 is Thinking
Philosopher 4 is Eating

Process returned 0 (0x0)   execution time : 0.083 s
Press any key to continue.
```

5.Write a C program to simulate:

a) Bankers' algorithm for the purpose of deadlock avoidance.
b) Deadlock Detection

a). Bankers' algorithm for the purpose of deadlock avoidance.

```
#include <stdio.h>
int main() {
    int n, r;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    printf("Enter number of resources: ");
    scanf("%d", &r);
    int alloc[n][r], max[n][r], avail[r], need[n][r], safeSeq[n];
    int finish[n];
    printf("Enter Allocation Matrix:\n");
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < r; j++) {
            scanf("%d", &alloc[i][j]);
        }
    }
    printf("Enter Maximum Demand Matrix:\n");
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < r; j++) {
            scanf("%d", &max[i][j]);
            need[i][j] = max[i][j] - alloc[i][j];
        }
    }
    printf("Enter Available Resources:\n");
    for (int i = 0; i < r; i++) {
        scanf("%d", &avail[i]);
    }
    for (int i = 0; i < n; i++) {
        finish[i] = 0;
    }
    int count = 0;
    while (count < n) {
        int found = 0;
        for (int p = 0; p < n; p++) {
            if (finish[p] == 0) {
                int j;
```

```c
                for (j = 0; j < r; j++) {
                    if (need[p][j] > avail[j])
                        break;
                }
                if (j == r) {
                    for (int k = 0; k < r; k++) {
                        avail[k] += alloc[p][k];
                    }
                    safeSeq[count++] = p;
                    finish[p] = 1;
                    found = 1;
                }
            }
        }
        if (found == 0) {
            printf("\nSystem is not in a safe state.");
            return 0;
        }
    }
    printf("\nSystem is in a safe state.\n");
    printf("Safe sequence is: ");
    for (int i = 0; i < n; i++) {
        printf("P%d", safeSeq[i]);
        if (i != n - 1) printf(" -> ");
    }
    printf("\n");
    return 0;
}
```

Output:

```
RIDHI GANESH

Enter number of processes: 5
Enter number of resources: 3
Enter Allocation Matrix:
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2
Enter Maximum Demand Matrix:
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3
Enter Available Resources:
3 3 2

System is in a safe state.
Safe sequence is: P1 -> P3 -> P4 -> P0 -> P2
Process returned 0 (0x0)   execution time : 49.122 s
Press any key to continue.
```

b). Deadlock Detection

```c
#include <stdio.h>
int main() {
    int n, r;
    printf("Enter the number of processes: ");
    scanf("%d", &n);
    printf("Enter the number of resources: ");
    scanf("%d", &r);
    int alloc[n][r], request[n][r], avail[r], safeSeq[n];
    int finish[n];
    printf("Enter the allocation matrix:\n");
    for (int i = 0; i < n; i++) {
        printf("Process %d: ", i);
        for (int j = 0; j < r; j++) {
            scanf("%d", &alloc[i][j]);
        }
```

```
    }
    printf("Enter the request matrix:\n");
    for (int i = 0; i < n; i++) {
        printf("Process %d: ", i);
        for (int j = 0; j < r; j++) {
            scanf("%d", &request[i][j]);
        }
    }
    printf("Enter the available resources: ");
    for (int i = 0; i < r; i++) {
        scanf("%d", &avail[i]);
    }
    for (int i = 0; i < n; i++) {
        finish[i] = 0;
    }
    int count = 0;
    while (count < n) {
        int found = 0;
        for (int p = 0; p < n; p++) {
            if (finish[p] == 0) {
                int j;
                for (j = 0; j < r; j++) {
                    if (request[p][j] > avail[j])
                        break;
                }
                if (j == r) {
                    for (int k = 0; k < r; k++) {
                        avail[k] += alloc[p][k];
                    }
                    safeSeq[count++] = p;
                    finish[p] = 1;
                    found = 1;
                }
            }
        }
        if (found == 0) {
            printf("\nSystem is not in a safe state.\n");
            return 0;
        }
```

```
    }
    printf("System is in safe state.\n");
    printf("Safe Sequence is: ");
    for (int i = 0; i < n; i++) {
        printf("P%d", safeSeq[i]);
        if (i != n - 1) printf(" ");
    }
    printf("\n");
    return 0;
}
```

Output:

```
ridhi ganesh
Enter number of processes: 5
Enter number of resources: 3
Enter allocation matrix:
Process 0: 0 1 0
Process 1: 2 0 0
Process 2: 3 0 3
Process 3: 2 1 1
Process 4: 0 0 2
Enter request matrix:
Process 0: 0 0 0
Process 1: 2 0 2
Process 2: 0 0 0
Process 3: 1 0 0
Process 4: 0 0 2
Enter available resources: 0 0 0
System is in safe state
Safe Sequence is: P0 P2 P3 P4 P1
Process returned 0 (0x0)   execution time : 47.398 s
Press any key to continue.
```

6. Write a C program to simulate the following contiguous memory allocation techniques.
a) Worst-fit b) Best-fit c) First-fit

```c
#include <stdio.h>
#define MAX 20
void firstFit(int blocks[], int m, int processes[], int n) {
    int allocation[MAX], occupied[MAX];
    for (int i = 0; i < n; i++) allocation[i] = -1;
```

```
    for (int i = 0; i < m; i++) occupied[i] = 0;
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            if (!occupied[j] && blocks[j] >= processes[i]) {
                allocation[i] = j;
                occupied[j] = 1;
                break;
            }
        }
    }
    printf("\n--- First Fit ---\n");
    printf("Process No.  Process Size  Block No.\n");
    for (int i = 0; i < n; i++) {
        printf("%d \t\t %d \t\t ", i + 1, processes[i]);
        if (allocation[i] != -1)
            printf("%d\n", allocation[i] + 1);
        else
            printf("Not Allocated\n");
    }
}
void bestFit(int blocks[], int m, int processes[], int n) {
    int allocation[MAX], occupied[MAX];
    for (int i = 0; i < n; i++) allocation[i] = -1;
    for (int i = 0; i < m; i++) occupied[i] = 0;
    for (int i = 0; i < n; i++) {
        int bestIdx = -1;
        for (int j = 0; j < m; j++) {
            if (!occupied[j] && blocks[j] >= processes[i]) {
                if (bestIdx == -1 || blocks[j] < blocks[bestIdx]) {
                    bestIdx = j;
                }
            }
        }
        if (bestIdx != -1) {
            allocation[i] = bestIdx;
            occupied[bestIdx] = 1;
        }
    }
    printf("\n--- Best Fit ---\n");
```

```c
    printf("Process No.  Process Size  Block No.\n");
    for (int i = 0; i < n; i++) {
        printf("%d \t\t %d \t\t ", i + 1, processes[i]);
        if (allocation[i] != -1)
            printf("%d\n", allocation[i] + 1);
        else
            printf("Not Allocated\n");
    }
}
void worstFit(int blocks[], int m, int processes[], int n) {
    int allocation[MAX], occupied[MAX];
    for (int i = 0; i < n; i++) allocation[i] = -1;
    for (int i = 0; i < m; i++) occupied[i] = 0;
    for (int i = 0; i < n; i++) {
        int worstIdx = -1;
        for (int j = 0; j < m; j++) {
            if (!occupied[j] && blocks[j] >= processes[i]) {
                if (worstIdx == -1 || blocks[j] > blocks[worstIdx]) {
                    worstIdx = j;
                }
            }
        }
        if (worstIdx != -1) {
            allocation[i] = worstIdx;
            occupied[worstIdx] = 1;
        }
    }
    printf("\n--- Worst Fit ---\n");
    printf("Process No.  Process Size  Block No.\n");
    for (int i = 0; i < n; i++) {
        printf("%d \t\t %d \t\t ", i + 1, processes[i]);
        if (allocation[i] != -1)
            printf("%d\n", allocation[i] + 1);
        else
            printf("Not Allocated\n");
    }
}
int main() {
    int m, n;
```

```c
    int blocks[MAX], processes[MAX];
    printf("Enter number of memory blocks: ");
    scanf("%d", &m);
    printf("Enter sizes of %d memory blocks:\n", m);
    for (int i = 0; i < m; i++) scanf("%d", &blocks[i]);
    printf("Enter number of processes: ");
    scanf("%d", &n);
    printf("Enter sizes of %d processes:\n", n);
    for (int i = 0; i < n; i++) scanf("%d", &processes[i]);
    firstFit(blocks, m, processes, n);
    bestFit(blocks, m, processes, n);
    worstFit(blocks, m, processes, n);
    return 0;
}
```

Output:

```
Enter number of memory blocks: 5
Enter sizes of 5 memory blocks:
100 500 200 300 600
Enter number of processes: 4
Enter sizes of 4 processes:
212 417 112 426

--- First Fit ---
Process No.     Process Size    Block No.
1               212             2
2               417             5
3               112             3
4               426             Not Allocated

--- Best Fit ---
Process No.     Process Size    Block No.
1               212             4
2               417             2
3               112             3
4               426             5

--- Worst Fit ---
Process No.     Process Size    Block No.
1               212             5
2               417             2
3               112             4
4               426             Not Allocated

Process returned 0 (0x0)   execution time : 20.815 s
Press any key to continue.
```

7. Write a C program to simulate page replacement algorithms.
a) FIFO
b) LRU
c) Optimal

```c
#include <stdio.h>
#include <stdbool.h>
void print_frames(int frames[], int count) {
    printf("[");
```

```
    for (int i = 0; i < count; i++) {
        printf("%d", frames[i]);
        if (i < count - 1) {
            printf(" ");
        }
    }
    printf("]\n");
}
void fifo(int pages[], int n, int f) {
    printf("\n--- FIFO Page Replacement ---\n");
    int frames[f];
    int count = 0;
    int next = 0;
    int faults = 0;
    for (int i = 0; i < n; i++) {
        int p = pages[i];
        bool hit = false;
        for (int j = 0; j < count; j++) {
            if (frames[j] == p) {
                hit = true;
                break;
            }
        }
        if (!hit) {
            if (count < f) {
                frames[count++] = p;
            } else {
                frames[next] = p;
                next = (next + 1) % f;
            }
            faults++;
        }
        printf("Page %d -> ", p);
        print_frames(frames, count);
    }
    printf("Total Page Faults (FIFO): %d\n", faults);
}
void lru(int pages[], int n, int f) {
    printf("\n--- LRU Page Replacement ---\n");
```

```
    int frames[f];
    int count = 0;
    int faults = 0;
    for (int i = 0; i < n; i++) {
        int p = pages[i];
        int hit_index = -1;
        for (int j = 0; j < count; j++) {
            if (frames[j] == p) {
                hit_index = j;
                break;
            }
        }
        if (hit_index != -1) {
            int val = frames[hit_index];
            for (int j = hit_index; j < count - 1; j++) {
                frames[j] = frames[j + 1];
            }
            frames[count - 1] = val;
        } else {
            if (count < f) {
                frames[count++] = p;
            } else {
                for (int j = 0; j < f - 1; j++) {
                    frames[j] = frames[j + 1];
                }
                frames[f - 1] = p;
            }
            faults++;
        }
        printf("Page %d -> ", p);
        print_frames(frames, count);
    }
    printf("Total Page Faults (LRU): %d\n", faults);
}
void optimal(int pages[], int n, int f) {
    printf("\n--- Optimal Page Replacement ---\n");
    int frames[f];
    int count = 0;
    int faults = 0;
```

```c
    for (int i = 0; i < n; i++) {
        int p = pages[i];
        bool hit = false;
        for (int j = 0; j < count; j++) {
            if (frames[j] == p) {
                hit = true;
                break;
            }
        }
        if (!hit) {
            if (count < f) {
                frames[count++] = p;
            } else {
                int farthest = -1;
                int replace_idx = -1;
                for (int j = 0; j < f; j++) {
                    int next_use = -1;
                    for (int k = i + 1; k < n; k++) {
                        if (frames[j] == pages[k]) {
                            next_use = k;
                            break;
                        }
                    }
                    if (next_use == -1) {
                        replace_idx = j;
                        break;
                    }
                    if (next_use > farthest) {
                        farthest = next_use;
                        replace_idx = j;
                    }
                }
                frames[replace_idx] = p;
            }
            faults++;
        }
        printf("Page %d -> ", p);
        print_frames(frames, count);
    }
```

```
    printf("Total Page Faults (Optimal): %d\n", faults);
}
int main() {
    int n, f;
    printf("Enter number of pages: ");
    scanf("%d", &n);
    int pages[n];
    printf("Enter page reference string:\n");
    for (int i = 0; i < n; i++) {
        scanf("%d", &pages[i]);
    }
    printf("Enter number of frames: ");
    scanf("%d", &f);
    fifo(pages, n, f);
    lru(pages, n, f);
    optimal(pages, n, f);
    return 0;
}
```

Output:

```
PS C:\Users\Pragati Tiwari\OneDrive\Desktop\OS Lab> & "C:\Users\Pragati Tiwari\OneDrive\Desktop\OS Lab\PageReplacement.exe"
Name: Pragati Tiwari || USN: 1WA24CS209
Enter number of pages: 12
Enter page reference string:
7 0 1 2 0 3 0 4 2 3 0 3
Enter number of frames: 3

--- FIFO Page Replacement ---
Page 7 -> [7]
Page 0 -> [7 0]
Page 1 -> [7 0 1]
Page 2 -> [2 0 1]
Page 0 -> [2 0 1]
Page 3 -> [2 3 1]
Page 0 -> [2 3 0]
Page 4 -> [4 3 0]
Page 2 -> [4 2 0]
Page 3 -> [4 2 3]
Page 0 -> [0 2 3]
Page 3 -> [0 2 3]
Total Page Faults (FIFO): 10

--- LRU Page Replacement ---
Page 7 -> [7]
Page 0 -> [7 0]
Page 1 -> [7 0 1]
Page 2 -> [0 1 2]
Page 0 -> [1 2 0]
Page 3 -> [2 0 3]
Page 0 -> [2 3 0]
Page 4 -> [3 0 4]
Page 2 -> [0 4 2]
Page 3 -> [4 2 3]
Page 0 -> [2 3 0]
Page 3 -> [2 0 3]
Total Page Faults (LRU): 9

--- Optimal Page Replacement ---
Page 7 -> [7]
Page 0 -> [7 0]
Page 1 -> [7 0 1]
Page 2 -> [2 0 1]
Page 0 -> [2 0 1]
Page 3 -> [2 0 3]
Page 0 -> [2 0 3]
Page 4 -> [2 4 3]
Page 2 -> [2 4 3]
Page 3 -> [2 4 3]
Page 0 -> [0 4 3]
Page 3 -> [0 4 3]
Total Page Faults (Optimal): 7
PS C:\Users\Pragati Tiwari\OneDrive\Desktop\OS Lab>
```